📊 Data handling & visualization

- **`import pandas as pd`**
  Used for working with structured data (tables).
  → Think Excel in Python (DataFrames).
- **`import numpy as np`**
  Handles numerical operations and arrays.
  → Faster math operations, matrix computations.
- **`import matplotlib.pyplot as plt`**
  Basic plotting library.
  → Line charts, bar charts, histograms.
- **`%matplotlib inline`**
  Jupyter Notebook magic command.
  → Makes plots display inside the notebook.
- **`import seaborn as sns`**
  Advanced visualization built on matplotlib.
  → Cleaner, more attractive statistical plots.
- **`pd.options.display.float_format = '{:.2f}'.format`**
  Formats numbers to 2 decimal places when displayed.

🧠 Feature selection

- **`SelectKBest`**
  Selects the top *K* most important features.
- **`chi2`**
  Statistical test used with `SelectKBest`
  → Measures relationship between categorical features and target.

🔀 Data splitting & validation

- **`train_test_split`**
  Splits dataset into training and testing sets.
- **`cross_val_score`**
  Performs cross-validation (more reliable model evaluation).
- **`RepeatedStratifiedKFold`**
  Advanced cross-validation technique
  → Keeps class balance while repeating splits.

🎯 Model evaluation metrics

- `confusion_matrix`
  Shows:
  - True positives
  - False positives
  - True negatives
  - False negatives
- `accuracy_score`
  Percentage of correct predictions.
- `classification_report`
  Detailed report including:
  - Precision
  - Recall
  - F1-score
- `roc_auc_score`
  Measures model's ability to distinguish classes (higher = better).
- `RocCurveDisplay`
  Plots ROC curve.
- `precision_recall_curve`
  Useful when dealing with imbalanced data.

⚙️ Model tuning

- `GridSearchCV`
  Finds best model parameters automatically.
  → Tries different combinations and picks the best.

🤖 Machine learning models

- `LogisticRegression`
  Used for binary classification (e.g., disease vs no disease).
- `DecisionTreeClassifier`
  Tree-based model that splits data into branches.
- **`SVC` (Support Vector Classifier)**
  Powerful model for classification, especially with complex boundaries.
- `RandomForestClassifier`
  Ensemble of decision trees → more accurate and robust.

- **`KNeighborsClassifier`**
  Classifies based on nearest data points.

🔧 Data preprocessing

- **`MinMaxScaler`**
  Scales values between 0 and 1.
- **`StandardScaler`**
  Standardizes data (mean = 0, std = 1).

⚠️ Warnings control

- **`warnings.filterwarnings('ignore')`**
  Hides warning messages (use carefully so you don't miss important issues).

🧠 Big picture (how this fits together)

You've essentially set up a **complete ML pipeline**:

1. Load data → `pandas`
2. Clean & scale → `scalers`
3. Select features → `SelectKBest`
4. Split data → `train_test_split`
5. Train models → (Logistic, RF, SVM, etc.)
6. Tune models → `GridSearchCV`
7. Evaluate → metrics + ROC + confusion matrix
8. Validate → cross-validation